📚 GIÁO TRÌNH: Hiểu Rõ Extension "VEO Automation" Từ A-Z

> **Dành cho học sinh phổ thông** — Viết bởi Giáo viên AI
> Extension: **VEO Automation - Auto VEO on Google Flow** | v2.3.9 | 713 KiB | ⭐ 4.6/5 (533 ratings)
> Developer: **Truong Nguyen** (kylenguyen.me) — Hà Nội, Việt Nam VN

📸 Giao Diện Thực Tế

```
![Side Panel chính – nơi điều khiển mọi thứ: chọn mode, nhập prompt, theo dõi tiến trì
<!— slide —>
![Queue Management – danh sách prompts đang xếp hàng chờ xử lý tự động](file:///C:/Use
<!— slide —>
![Trang Chrome Web Store của extension](file:///C:/Users/intel/.gemini/antigravity/bra
```

MỤC LỤC

Chương	Nội dung	Mức độ
I	Extension là gì? Tại sao cần nó?	🟢 Cơ bản
II	VEO Automation làm gì? — Bức tranh tổng quan	🟢 Cơ bản
III	Kiến trúc 4 tầng — "Bộ máy" bên trong	🟡 Trung bình
IV	manifest.json — "Giấy khai sinh" của extension	🟡 Trung bình
V	Service Worker — "Bộ não" điều phối	🔴 Nâng cao
VI	Content Script — "Tay robot" thao tác DOM	🔴 Nâng cao
VII	MAIN World Injection — "Gián điệp" bắt API	🔴 Nâng cao

Chương	Nội dung	Mức độ
VIII	Side Panel UI — Bảng điều khiển cho người dùng	🟡 Trung bình
IX	10 kỹ thuật "bí kíp" quan trọng nhất	🔴 Nâng cao
X	Cách xuất bản lên Chrome Web Store	🟢 Cơ bản
XI	Bài tập và câu hỏi ôn tập	📝 Thực hành

CHƯƠNG I: Extension Là Gì? — Kiến Thức Nền Tảng

1.1 Định nghĩa đơn giản

> **Chrome Extension** = một chương trình nhỏ được cài thêm vào trình duyệt Chrome, giúp Chrome có thêm tính năng mới.

Ví dụ dễ hiểu:

- Chrome giống như chiếc **điện thoại** của bạn
- Extension giống như **ứng dụng (app)** bạn cài thêm
- Cài app mới → điện thoại làm được thêm nhiều thứ

1.2 Extension khác Website như thế nào?

Đặc điểm	Website	Extension
Chạy ở đâu?	Trong 1 tab	Bên ngoài tab, "bao bọc" tất cả tabs
Quyền truy cập	Chỉ trang web đó	Có thể sửa đổi BẤT KỲ trang web nào
Lưu dữ liệu	Cookie, localStorage	`chrome.storage` (riêng biệt, bảo mật hơn)
Giao tiếp	Chỉ nội bộ	Có thể giao tiếp giữa nhiều thành phần
Cài đặt	Không cần	Qua Chrome Web Store hoặc file ZIP

1.3 Manifest V3 — Phiên bản mới nhất

Chrome yêu cầu mọi extension phải tuân theo **Manifest V3** (từ 2024):

- **V2** (cũ): Dùng "background page" — chạy liên tục, tốn RAM

- **V3** (mới): Dùng "service worker" — chỉ chạy khi cần, tiết kiệm tài nguyên

> ℹ️ **NOTE**
>
> VEO Automation dùng **Manifest V3** — phiên bản hiện đại nhất. Nếu bạn tự làm extension, BẮT BUỘC phải dùng V3.

CHƯƠNG II: VEO Automation Làm Gì? — Bức Tranh Tổng Quan

2.1 Vấn đề cần giải quyết

Google Flow (labs.google.com/fx/tools/flow) là công cụ AI của Google cho phép tạo video/ảnh từ text prompt. **Nhưng có nhược điểm:**

- Chỉ tạo được **1 video tại 1 thời điểm**
- Phải **nhập prompt thủ công** từng cái
- Phải **chờ xong rồi mới tải về** bằng tay
- Nếu muốn tạo 50 video → mất **hàng giờ click chuột**

2.2 Giải pháp của VEO Automation

Extension biến Google Flow thành **"nhà máy sản xuất video tự động"**:

```
TRƯỚC (thủ công):                SAU (có extension):
─────────────────                ───────────────────
1. Mở Google Flow                1. Dán 50 prompts vào Side Panel
2. Nhập prompt #1                2. Bấm ▶️ Start
3. Chọn model, ratio             3. Đi làm việc khác ☕
4. Bấm Generate                  4. Quay lại → 50 video đã tải xong!
5. Chờ 2-5 phút ...
6. Tải video
7. Lặp lại 49 lần nữa 😫
```

2.3 Các chế độ hoạt động (6 modes)

Mode	Input	Output	Giải thích
Text to Video	Đoạn văn bản	Video AI	Viết mô tả → AI tạo video
Frame to Video	Ảnh + văn bản	Video AI	Dùng ảnh làm khung hình đầu → AI tạo video tiếp

Mode	Input	Output	Giải thích
Ingredients to Video	Nhiều ảnh + text	Video AI	Kết hợp nhiều yếu tố → video
Text to Image	Đoạn văn bản	Ảnh AI	Viết mô tả → AI tạo ảnh
Image to Image	Ảnh + văn bản	Ảnh AI mới	Biến đổi ảnh theo mô tả
Last Image to Image	Ảnh cuối cùng + text	Ảnh AI mới	Dùng ảnh vừa tạo xong làm input cho lần tiếp

2.4 Tính năng nổi bật

- **Auto-Download**: Tự động tải video/ảnh khi xong (720p, 1080p, 2K, 4K)
- **Auto-Chaining**: Lấy frame cuối video trước → làm ảnh tham chiếu cho video sau
- **Concurrent Processing**: Chạy song song nhiều job cùng lúc (tối đa 2)
- **Queue Management**: Xếp hàng hàng trăm prompts, theo dõi tiến trình realtime
- **6 ngôn ngữ**: English, Tiếng Việt, 中文, 한국어, 日本語, Español

CHƯƠNG III: Kiến Trúc 4 Tầng — "Bộ Máy" Bên Trong

3.1 Tổng quan kiến trúc

> ❗ **IMPORTANT**
>
> Đây là kiến thức **QUAN TRỌNG NHẤT** để hiểu extension. Extension có **4 tầng**, mỗi tầng đảm nhận vai trò khác nhau, giống như một công ty có 4 phòng ban.

📊 Diagram

```
graph TB
    subgraph "🧩 Extension VEO Automation (713 KiB)"
        direction TB
        MF["📄 manifest.json<br/>(Giấy khai sinh)"]

        subgraph "Tầng 1: 🖥️ GIAO DIỆN (UI Layer)"
            SP["Side Panel<br/>panel.html + panel.js + panel.css<br/>• Nơi người dùng NHÌN
THẤY và TƯƠNG TÁC<br/>• Nhập prompts, chọn cài đặt, xem tiến trình"]
        end
```

```
        subgraph "Tầng 2: ⚙️ ĐIỀU PHỐI (Orchestration Layer)"
            SW["Service Worker<br/>service-worker.js<br/>• BỘ NÃO — không có giao diện!
<br/>• Quản lý hàng đợi (queue)<br/>• Lên lịch chạy jobs<br/>• Điều phối tải video"]
        end

        subgraph "Tầng 3: 🚀 TỰ ĐỘNG HÓA (Automation Layer)"
            CS["Content Script<br/>flow-automator.js<br/>• TAY ROBOT — thao tác trên Google
Flow<br/>• Click nút, điền prompt, chọn model<br/>• Theo dõi tiến trình rendering"]
            MAIN["MAIN World Script<br/>page-interceptor.js<br/>• GIÁN ĐIỆP — bắt dữ liệu
API<br/>• Chặn fetch() để lấy URL video<br/>• Hoạt động trong cùng thế giới JS với Google
Flow"]
        end

        subgraph "Tầng 4: 💾 DỮ LIỆU (Data Layer)"
            STORE["chrome.storage.local<br/>• Lưu trạng thái queue<br/>• Lưu cài đặt người
dùng<br/>• Persist qua restart"]
            I18N["_locales/<br/>• en, vi, zh, ko, ja, es<br/>• Đa ngôn ngữ"]
        end
    end

    SP <-->|"chrome.runtime.sendMessage()"| SW
    SW <-->|"chrome.tabs.sendMessage()"| CS
    CS -->|"Inject vào trang"| MAIN
    SW <-->|"get/set"| STORE

    style SP fill:#1a1a2e,stroke:#e94560,color:#fff
    style SW fill:#16213e,stroke:#0f3460,color:#fff
    style CS fill:#0f3460,stroke:#533483,color:#fff
    style MAIN fill:#533483,stroke:#e94560,color:#fff
```

3.2 Ví dụ dễ hiểu — So sánh với nhà hàng

Tầng Extension	Vai trò trong nhà hàng	Giải thích
Side Panel (UI)	📋 **Bồi bàn**	Nhận order từ khách (prompts), thông báo trạng thái
Service Worker	👨‍🍳 **Quản lý bếp**	Phân công, sắp xếp thứ tự nấu, không trực tiếp nấu
Content Script	🍳 **Đầu bếp**	Trực tiếp thao tác — nhập liệu, bấm nút, chờ kết quả
MAIN World	🕵️ **Người giao hàng**	Bắt video URL từ Google API → chuyển về
chrome.storage	📝 **Sổ ghi chép**	Ghi lại đơn hàng, không bị mất khi tắt bếp

3.3 Luồng giao tiếp — Ai nói chuyện với ai?

```
📊 Diagram
```

```
sequenceDiagram
    participant U as 👤 Người dùng
    participant SP as 🖥️ Side Panel
    participant SW as ⚙️ Service Worker
    participant CS as 💉 Content Script
    participant MAIN as 🔓 MAIN World
    participant GF as 🌐 Google Flow

    U->>SP: Dán 50 prompts + bấm Start
    SP->>SW: sendMessage({action: 'ADD_JOBS', prompts: [...]})
    SW->>SW: Tạo queue 50 jobs

    loop Với mỗi job (tối đa 2 song song)
        SW->>CS: sendMessage({action: 'EXECUTE_AUTOMATION', job})
        CS->>GF: Click "New Project"
        CS->>GF: Điền prompt vào textarea
        CS->>GF: Click "Generate"
        GF->>MAIN: fetch() tới Google API
        MAIN->>CS: CustomEvent('__VEO_VIDEO_URL__', {url})
        CS->>SW: sendResponse({success, videoUrl})
        SW->>SW: chrome.downloads.download(videoUrl)
        SW->>SP: sendMessage({event: 'QUEUE_UPDATE'})
        SP->>U: Cập nhật thanh tiến trình ✅
    end
```

CHƯƠNG IV: manifest.json — "Giấy Khai Sinh" Của Extension

4.1 File này là gì?

> **manifest.json** là file **BẮT BUỘC** của mọi Chrome Extension. Nó khai báo:
>
> - Extension tên gì, version bao nhiêu
> - Cần những **quyền** gì (permissions)
> - Được chạy trên **website nào**
> - Có những **file nào** (background, content scripts, UI)

4.2 Phân tích từng dòng

```
{
  // ═══ PHẦN 1: THÔNG TIN CƠ BẢN ═══

  "manifest_version": 3,
  // ↑ BẮT BUỘC là 3. Chrome không chấp nhận V2 nữa.
  // Giống như khai "phiên bản quy chế" khi làm giấy tờ.
```

```
  "name": "__MSG_extensionName__",
  // ↑ __MSG_xxx__ = ký hiệu đặc biệt → Chrome tự tìm trong thư mục _locales/
  // Nếu browser đang dùng tiếng Việt → tìm _locales/vi/messages.json
  // Nếu không có → dùng default_locale (English)

  "version": "2.3.9",
  // ↑ SemVer: Major.Minor.Patch
  // Major=2 (thay đổi lớn), Minor=3 (tính năng mới), Patch=9 (sửa lỗi)

  "default_locale": "en",
  // ↑ Ngôn ngữ mặc định. BẮT BUỘC khi dùng __MSG_xxx__

  // ═══ PHẦN 2: PERMISSIONS — Xin quyền ═══

  "permissions": [
    "sidePanel",    // Quyền mở bảng điều khiển bên cạnh trang web
    "storage",      // Quyền lưu dữ liệu (queue, settings) — KHÔNG phải cookies!
    "downloads",    // Quyền tự động tải file (video/ảnh)
    "scripting",    // Quyền chèn script vào trang web đang mở
    "tabs",         // Quyền xem danh sách tab và gửi tin nhắn đến tab
    "activeTab"     // Quyền truy cập tab đang active khi user click icon
  ],
  // ↑ Chrome sẽ hỏi người dùng: "Extension này xin các quyền sau..."
  // Người dùng phải đồng ý mới cài được.

  // ═══ PHẦN 3: HOST PERMISSIONS — Chạy trên web nào? ═══

  "host_permissions": [
    "https://labs.google.com/*",
    "https://aisandbox-pa.googleapis.com/*"
  ],
  // ↑ Extension CHỈ hoạt động trên 2 domain này.
  // Không thể đọc/sửa Facebook, Gmail, hay bất kỳ trang nào khác.
  // → Bảo mật tốt! (Nguyên tắc "Least Privilege")

  // ═══ PHẦN 4: BACKGROUND — Bộ não ═══

  "background": {
    "service_worker": "background/service-worker.js",
    "type": "module"  // Cho phép dùng import/export hiện đại
  },
  // ↑ Service Worker = chương trình chạy ngầm
  // KHÔNG CÓ GIAO DIỆN — không thể dùng document, window
  // Chrome TỰ ĐỘNG TẮT sau 30 giây không hoạt động (tiết kiệm RAM)

  // ═══ PHẦN 5: CONTENT SCRIPTS — Tay robot ═══

  "content_scripts": [{
    "matches": ["https://labs.google.com/fx/tools/flow/*"],
    "js": ["content-scripts/flow-automator.js"],
    "run_at": "document_idle"
  }],
  // ↑ matches: Chỉ inject vào Google Flow (không phải mọi trang)
  // ↑ run_at: "document_idle" = chạy SAU KHI trang đã load xong
  //    Alternatives: "document_start" (trước DOM), "document_end" (sau DOM, trước ima
```

```
  // ═══ PHẦN 6: SIDE PANEL - Giao diện người dùng ═══

  "side_panel": {
    "default_path": "side-panel/panel.html"
  },
  // ↑ File HTML hiển thị trong bảng bên cạnh
  // Panel này PERSISTENT - không bị đóng khi chuyển tab!

  // ═══ PHẦN 7: ICONS - Biểu tượng ═══

  "action": {
    "default_icon": { "16": "icons/icon16.png", "32": "icons/icon32.png" },
    "default_title": "__MSG_extensionName__"
  },
  "icons": {
    "16": "icons/icon16.png",    // Favicon nhỏ
    "48": "icons/icon48.png",    // Trang quản lý extension
    "128": "icons/icon128.png"   // Chrome Web Store (con hamster 🐹)
  }
}
```

> 💡 **TIP**
>
> **Bài học cho học sinh:** Mỗi permission là một "lời hứa" với người dùng. Extension xin CÀNG ÍT quyền CÀNG TỐT → người dùng tin tưởng hơn → nhiều người cài hơn.

CHƯƠNG V: Service Worker — "Bộ Não" Điều Phối

5.1 Service Worker là gì?

Ví dụ: Service Worker giống như **tổng đài viên** ngồi trong phòng kín:

- **Không nhìn thấy** gì bên ngoài (không có DOM, không có `document`)
- Chỉ **nghe điện thoại** (nhận messages) và **gọi điện** (gửi messages)
- Có thể **ghi chú** (chrome.storage) để nhớ việc
- Chrome **tắt máy** của tổng đài viên sau 30s không có cuộc gọi → tiết kiệm "lương"

5.2 Vòng đời (Lifecycle)

```
Cài extension lần đầu
        ↓
chrome.runtime.onInstalled → Set default settings
        ↓
```

```
Service Worker IDLE (ngủ) ← Chrome terminate sau 30s
        ↓
Có message/alarm/event → Chrome khởi động lại SW
        ↓
Xử lý xong → Quay lại IDLE → Bị terminate ...
        ↓
(Vòng lặp tiếp tục)
```

5.3 QueueManager — Hệ thống hàng đợi

```
class QueueManager {
  constructor() {
    this.queue = [];          // ← Jobs chờ xử lý (FIFO: vào trước ra trước)
    this.running = new Map(); // ← Jobs đang chạy (tối đa 2)
    this.completed = [];      // ← Jobs đã hoàn thành ✅
    this.failed = [];         // ← Jobs bị lỗi ❌
    this.maxConcurrent = 2;   // ← "Semaphore" — giới hạn song song
    this.isPaused = false;    // ← Có đang tạm dừng không?
  }
}
```

Giải thích FIFO + Semaphore:

```
Queue (hàng đợi FIFO):     Running (đang chạy):    Completed:
┌──────────────────┐       ┌────────────────┐      ┌──────────┐
│ Job 5 (chờ)      │   ┌── │ Job 3 (45%)    │  ┌──→ │ Job 1 ✅│
│ Job 4 (chờ)      │   │   │ Job 2 (72%)    │  │    │ Job 0 ✅│
│ Job 3 ──────────────┘    └────────────────┘  │    └──────────┘
│                  │       maxConcurrent = 2    │
└──────────────────┘       (tối đa 2 cùng lúc) │
                           Khi 1 job xong ──────┘ → lấy job tiếp từ queue
```

5.4 Luồng xử lý processNext()

```
async processNext() {
  // Bước 1: Kiểm tra điều kiện
  if (this.isPaused) return;                    // Đang tạm dừng → không làm gì
  if (this.running.size ≥ this.maxConcurrent) return;  // Đã đủ 2 jobs → chờ
  if (this.queue.length === 0) return;          // Hết việc → nghỉ

  // Bước 2: Lấy job ĐẦU TIÊN trong queue (FIFO)
  const job = this.queue.shift();  // shift() = lấy phần tử đầu và XÓA khỏi mảng
  job.status = 'running';
  this.running.set(job.id, job);

  // Bước 3: Auto-Chaining (nối video liên tục)
  // Lấy frame cuối của video trước → làm ảnh tham chiếu cho job này
  if (this.completed.length > 0) {
    const lastJob = this.completed[this.completed.length - 1];
```

```
    if (lastJob.lastFrameUrl) {
      job.referenceImage = lastJob.lastFrameUrl;
    }
  }

  // Bước 4: Tìm tab Google Flow và gửi lệnh xuống Content Script
  const tabs = await chrome.tabs.query({ url: 'https://labs.google.com/fx/tools/flow/*
  const result = await chrome.tabs.sendMessage(tabs[0].id, {
    action: 'EXECUTE_AUTOMATION',
    payload: job
  });

  // Bước 5: Xử lý kết quả
  if (result.success) {
    job.status = 'completed';
    this.completed.push(job);
    // Tự động tải video
    await chrome.downloads.download({ url: result.videoUrl, filename: '...' });
  }

  // Bước 6: LƯU STATE + gọi processNext() lần nữa (đệ quy)
  await this.persistState();  // Lưu vào chrome.storage
  this.broadcastUpdate();     // Thông báo UI cập nhật
  this.processNext();         // ← ĐÂY! Tự gọi lại chính mình → xử lý job tiếp
}
```

> ⚠️ **WARNING**
>
> **Bẫy quan trọng:** Service Worker BỊ CHROME TẮT sau 30 giây không hoạt động! Nếu không lưu state vào `chrome.storage`, mọi dữ liệu queue sẽ **MẤT SẠCH** khi SW bị tắt rồi bật lại. Đó là lý do có hàm `persistState()` và `restoreState()`.